

Spatial Distribution of bidirectional Charging Stations by Simulation and Evaluation of Traffic and Grid Data

Achim Baumgärtner, Erik Weigand

July 2025



Nah an Mensch und Technik.



Use of AI Tools

In the course of this academic work, the AI tools ChatGPT (GPT-4o, OpenAI) and DeepL Write were utilized to enhance translation, linguistic clarity, expression, and formatting. The tool was primarily used to enhance language and formatting, without generating or contributing to the academic content itself.

Specifically, AI assistance was applied during the writing of the following sections: *Abstract*, *Research (Presentation of Relevant Tools)*, *Discussion (Further Work)*. In these sections, the tool was used exclusively to refine grammar, style, and readability.

Given that the thesis was written using LaTeX, ChatGPT was also used to support formatting tasks, such as generating tables. The data used in these tables was originally created in Microsoft Excel and later formatted accordingly.

During the early research phase, ChatGPT was consulted to clarify basic concepts and terminology, supporting initial orientation and comprehension.

All substantive research, content development, analysis, and interpretation were carried out by the authors independently, based on personal work and literature review.

Declaration of Academic Integrity

We hereby declare that we have written the present work independently and have used no sources or aids other than those indicated.

All passages taken directly or indirectly from other sources have been clearly marked.

Waiblingen, August 28, 2024

A stylized, cursive handwritten signature in black ink, appearing to read 'AB'.

(Signature)

Achim Baumgärner

A handwritten signature in blue ink that reads 'Erik Weigand' in a cursive script.

(Signature)

Erik Weigand

Abstract

This work focuses on the research, development, and prototypical implementation of a simulation-based system for planning and optimizing electric vehicle (EV) charging infrastructure in urban environments. Despite recent advances in e-mobility and bidirectional charging, there remains a lack of data-driven tools for the strategic placement of charging stations.

To address this gap, a flexible simulation workflow was designed using OpenStreetMap data and SUMO to generate realistic traffic flows and EV charging behaviors via custom Python scripts. An initial approach combining MATSim and SUMO was discarded due to technical challenges.

The core of the system is an iterative optimization algorithm that evaluates real-time charging demand. Using TraCI, vehicle behavior and charging states are monitored and logged. Underused stations are systematically removed, leading to a more efficient spatial distribution of stations.

The anticipated outcome is a web-based planning tool that helps urban planners and grid operators to simulate realistic scenarios, adjust parameters, and identify optimal locations for EV charging stations. In the future, the system may be expanded to support grid integrations and vehicle-to-grid (V2G) strategies in order to meet the demands of future mobility and energy infrastructures.

Inhaltsverzeichnis

Abbildungsverzeichnis	vi
List of Abbreviations	vii
1 Introduction	1
1.1 Motivation	1
1.2 Relevance	1
2 Research Objectives	3
2.1 Research goals	3
2.2 Research phases	3
2.3 Underlying assumptions	4
2.4 Anticipated contributions	5
3 Methodology	6
4 Requirements	7
4.1 Requirements	7
4.2 Success Criteria	9
5 Research	10
5.1 Realistic Traffic Scenario Generation	10
5.1.1 Requirements for Realistic Traffic Scenario Generation	10
5.1.2 Presentation of relevant tools	11
SUMO	11
TraCi	12
SAGA	12
MATSim	13
BEAM	13
JOSM	14
5.1.3 Combination of SUMO with different frameworks/ tools	14
Combination of SAGA and SUMO	14
Combination of MATSim and SUMO	15
Combination of BEAM and SUMO	15
5.1.4 Comparison of introduced concepts	16
5.2 Charging Station Placement Algorithm	19
5.2.1 Requirements for Charging Station Placement Algorithm	19
5.2.2 Assumptions	19
Parking areas	19
Points of Interests	19

5.2.3	Charging Station Algorithm	20
5.3	Integration of Grid Models	21
5.3.1	Requirements for Integration of Grid Models	21
5.3.2	Approaches to integrate V2G functionality	21
	Integration of Power Lines into Simulation Network	21
	Runtime Monitoring of Grid Loads	21
6	Decision and implementation	23
6.1	Decision	23
6.2	Implementation	23
6.2.1	Prototyping	23
6.2.2	Refactoring and technical learning	24
6.2.3	Current implementation	24
6.2.4	Overview of architecture and Dataflow	25
	Population Generation	26
	Charging Station Generation	28
	Simulation	29
7	Results	31
7.1	Showcase	31
7.2	Functionalities	31
8	Discussion	34
8.1	Interpretation of Results and Evaluation of Workflows	34
8.2	Optimizing Charging Station Placement	34
8.3	Challenges with Power Grid Integration	35
8.4	Fulfillment of Research Objectives	35
8.5	Further work	35

Abbildungsverzeichnis

1	Integration of MATSim & SUMO	15
2	Integration of BEAM & SUMO	16
3	Comparison of Heatmaps for Roads and Residency	20
4	Flowchart of Prototype Design	24
5	Flowchart of Creating and Running a Simulation	25
6	Abstracted View of Architecture	26
7	Trip Generation Activity Diagram	27
8	Charging Station Deployment Script Activity Diagram	28
9	System Activity Diagram	29
10	Showcase of current implementation	31

List of Abbreviations

AC Alternating Current

OSM OpenStreetMap

V2G Vehicle-to-Grid

1 Introduction

The number of electric vehicles and charging infrastructure in Germany has grown substantially in recent years. Today, there are more than 1.6 million battery-operated vehicles on German roads, with more joining every day. Alongside the increase in EVs, the charging network is becoming denser too, ranging from fast charging stations on motorways to Alternating Current (AC) stations in city centers and shopping centers **noauthor' elektroauto-statistiken' no date**.

However, simply increasing the number of charging stations will not be enough. The locations and integration into the power grid must be thoroughly planned to, for example, balance peak and off-peak loads and optimize user needs.

The German government plans to have around 15 million electric vehicles on German roads by 2030, and aims to provide around one million public charging stations by then **klimaschutz' rahmenbedingungen**. To actually realize that economically and ecologically, concepts like intelligent locations are becoming more important.

The next step in innovation would be bidirectional charging (Vehicle-to-Grid (V2G)). This would mean that electric vehicles would not only be a means of transportation, but also a form of mobile energy storage that could feed surplus power back into the grid. This would help to cushion load peaks and increase the stability of the overall system, making an important contribution to the energy transition, especially in combination with volatile renewables.

These developments clearly show that simply increasing the number of charging stations is not enough. It is becoming increasingly important to integrate these technologies in a smart way in order to promote electric mobility sustainably and across the board.

1.1 Motivation

Reliably testing and validating the positions of charging stations in a controlled environment would be a significant step toward an automated ecosystem in which companies or government entities could respond to demand. In short, the goal is to develop a system that improves overall efficiency and effectiveness while producing meaningful and realistic results.

1.2 Relevance

Simulation tools can be used to recreate realistic traffic and charging behavior in a virtual environment, enabling testing different locations and operational strategies at an early stage. By modeling driving patterns and charging workflows, it is possible to analyze typical routes and charging patterns. This allows charging stations to be placed where they will be most beneficial and minimize no-load times. In addition, grid loads can be estimated in advance to avoid overloads and enable expansion where the power grid still has capacity.

In terms of bidirectional charging, simulation tools can help combine driver behavior and network

analysis. This makes it possible to calculate how much energy can be fed back into the grid without creating voltage peaks. Different scenarios can be simulated, such as the bulk loading that occurs when people get home from work and connect their cars to their home charging stations.

Using simulation to support planning processes can ensure greater reliability and efficiency by providing data-based insights into optimal locations, performance, and network requirements. Eliminating the need for on-site tests and pilot systems can significantly reduce planning and testing costs. Simulation forms the basis of smart, well-thought-out charging infrastructure.

2 Research Objectives

The continuous growth of e-mobility in urban areas presents city planners and grid operators with new challenges. While technologies such as V2G open up new possibilities, they also require precise planning of locations. The following chapter outlines the overarching objectives of this work and breaks them down into successive phases. Technical implementation details will not be discussed yet.

2.1 Research goals

The goal of this research is to develop a framework that optimizes the location and validates the placement of bidirectional charging stations under realistic traffic conditions.

2.2 Research phases

The research is organized into seven phases, each with specific tasks and deliverables. The phases span from a foundational literature review and prototype development to V2G integration, scalable deployment, and a final interactive dashboard.

Phase 1: Basics and literature review
• Derive functional & non-functional requirements • Define evaluation metrics (coverage, grid load, wait times) • Research methods & data sources: – OpenStreetMap (OSM) – Simulation tools – Traffic scenarios – Charging behavior – Grid models – Charging Station Placement algorithms – V2G concepts Deliverable: requirements, concept (Workflow)
Phase 2: Prototypical implementation
• Traffic scenario • Algorithm to determine best locations • Placement of first charging stations Deliverable: example map with prototype algorithm
Phase 3: Validation
• Iterative Validation • Use defined metrics to evaluate placement of charging stations Deliverable: report

Phase 4: Vehicle-to-Grid Implementation
• Data sources • Implementation into simulation + algorithm Deliverable: example map with prototype algorithm + V2G
Phase 5: Generalization & scaling
• Implement for arbitrary maps/ regions • Optimize performance Deliverable: application for arbitrary maps
Phase 6: Web Application
• Frontend for map upload & parameter selection • Backend service for map handling and algorithm execution Deliverable: working web application
Phase 7: Dashboard
• Interactive dashboard (map + KPIs) • Export options (CSV, GeoJSON) Deliverable: app with live-dashboard

The project is designed to span three semesters. Phases 1 and 3 are scheduled for the first semester. Phases 4 and 5 are planned for the second semester, and phases 6 and 7 are planned for the third and final semester.

Although SUMO is not a mandatory requirement for creating a realistic traffic scenario or determining the locations for the charging stations, it will be used for running the simulation itself. For that reasons the workflow should incorporate it as much into the workflow as possible, especially to ensure backwards compatibility.

2.3 Underlying assumptions

The following assumptions form the basis for the modeling and simulation of traffic and charging processes.

- **Homogeneous Fleet Parameters:** All electric vehicles do have the same charging power and battery capacity to reduce simulation complexity. This can change depending on the progress of this project.
- **Data Quality:** OpenStreetMap geometries and the network data are sufficiently complete and error-free.

2.4 Anticipated contributions

This section outlines the anticipated key contributions of this research:

- **Web Application and Dashboard for planners:** intuitive map upload, fine-tuning of parameters, interactive visualization, and reporting.
- **Concrete recommendations:** for city planners and grid operators regarding bidirectional charging infrastructure.

3 Methodology

The following section describes the methodological approach used to research and develop a system to evaluate and optimize the distribution of electric charging stations using traffic simulation. The aim was to optimize locations based on synthetic traffic data and different placement algorithms.

Here, an exploratory approach was chosen with the goal of identifying and evaluating potential locations for charging stations in iterative test series. The central research questions were:

- How can you simulate traffic as realistically as possible using SUMO?
- What does an algorithm for optimizing the location of charging stations in urban traffic look like?
- Which placement strategy (random vs. based on traffic volume) minimizes the number of empty loading cycles?
- How can charging locations be evaluated?

Various OSM map areas of different sizes served as the basis for development. For the first steps, parameters such as people, daily schedules, and the number of initial charging stations were set arbitrarily. This initially served to develop a stable workflow.

For the optimization algorithm itself an iterative development strategy was applied. Charging stations were initially placed based on heuristics (e.g. traffic density, lane length or randomly), and each simulation cycle logged charging behaviors. The results informed subsequent iterations, aiming to minimize empty charging cycles and improve station utilization.

A variety of tools and frameworks were utilized throughout the course of this work to build, simulate, and evaluate traffic scenarios. The simulation regions were obtained using JSOM (v19369) and osmGet (provided by SUMO), both of which provide raw OSM data. These were then used as inputs and further processed by the following tools.

SUMO (v1.24.0) was selected as the primary simulation framework due to its robust ecosystem of tools (e.g. `netconvert`, `duarouter`, `osmGet`, and `osmWebWizard`) and its support for detailed, customizable traffic scenarios. Its visual interface and Python integration via TraCI (v1.24.0) allowed for real-time parameter adjustments and dynamic data collection.

Initially, MATSim (v2024.0) was used for generating realistic daily schedules, particularly due to its strength in simulating detailed agent behavior. However, it was eventually phased out due to inconsistent outputs, unreliable conversion scripts, and lack of determinism. It was replaced by custom Python (v3.13.2) scripts, which provided greater reliability, transparency, and control over scenario generation.

All configuration files and scripts were maintained under version control via Git and GitHub. A structured project board tracked ongoing, completed, and planned features. To minimize outliers and guarantee determinism, simulations were run multiple times.

4 Requirements

In the context of software, requirements state boundaries or necessities of specific circumstances or events that occur, so that the software is still able to run without any inconvenience. Requirements in the field of product ownership generally means what exactly the software does and what not. The requirements get communicated mainly with the customers and the product owner, who is trained to figure out the exact needs for the software. Below, the general requirements for the system are first defined in functional and non-functional terms. Then, the specific requirements for the core components are presented in detail.

4.1 Requirements

1. Overall System Requirements			
ID	Name	Description	Priority
R1.1	OSM-Integration	The system must be able to integrate maps from OpenStreetMap.	High
R1.2	Automation	The system must enable the developed workflow (network & traffic generation, simulation, analysis, output) to be automated and script-controlled.	High
R1.3	Configurable Parameters	Users have to be able to set parameters like charging power, number of charging stations and agents, mobility patterns, charging behavior profiles and optionally power grid data.	Medium
R1.4	Modularity	The system should have a modular structure so that components such as simulation, traffic generation and visualization can be processed independently.	Low
R1.5	Extensibility	The system architecture should support future extensions (e.g. new optimization algorithms, data inputs).	Low
R1.6	Performance	The system should handle large geographic areas and thousands of agents in < 10 minutes.	High
R1.7	Usability	The web UI should be intuitive, with real-time feedback and helpful defaults.	Medium

R1.8	Reproducibility	The system must allow reproducibility via random seeds, logs, and versioning.	High
2. Traffic and Charging Behavior			
R2.1	EVs with Charging Profiles	The system must simulate electric vehicles with energy-related behavior (SOC, charging-demand, consumption).	High
R2.2	Charging Station Placement	It must be possible to define charging points in the network via script and independently of main traffic.	High
R2.3	Bottleneck Detection	The system should detect bottlenecks at charging points.	Medium
R2.4	Traffic Agent Generation	The system should generate realistic agents with daily routines (e.g. work-home-leisure), including rush hours.	High
R2.5	Vehicle Configuration	It must support agent density, battery sizes, vehicle types.	Medium
3. Location Analysis and Optimization			
R3.1	Clustering Analysis	Hotspot analysis for EV charging demand.	High
R3.2	New Charging Point Proposals	Automatic suggestion of new charging locations.	High
R3.3	Data Export	Export of results (GeoJSON, XML).	Medium
R3.4	Multi-Criteria Optimization	Optimize locations based on traffic flow, accessibility, grid capacity.	High
R3.5	Optimization Goals	Support goals like maximizing coverage or minimizing average distance.	Medium
R3.6	Result Scoring	Provide ranked list of candidate sites with scores.	Medium
R3.7	Logs	Maintain detailed logs for all optimization runs.	Medium
4. Network Modeling and Geo-data Processing			
R4.1	Automatic conversion	Convert OpenStreetMap into simulation-ready format.	High
R4.2	POIs	Automatically detect POIs from OSM data.	High

R4.3	Network	Automatically generate simulation-ready network (.xml).	High
R4.4	Grid Data Support	Optionally integrate power grid data into optimization.	Medium
5. Population Behavior			
R5.1	Daily Patterns	The population must follow a reasonable daily pattern (home-work-leisure-home-patterns).	High
R5.2	Temporal Variation	Include stochastic variation in departure/arrival times.	Medium
R5.3	Population	Automatically generate simulation-ready population (.pop).	High
6. Visualization and Validation			
R6.1	SUMO Integration	Calculated stations must be integrated in SUMO for validation.	High
R6.2	Simulation Metrics	Collect charging behavior, grid load, waiting times.	High
R6.3	Interactive Dashboard	Display station locations, KPIs, simulation results, export (CSV, GeoJSON).	Medium

4.2 Success Criteria

The system is successful if users can:

1. Import arbitrary OSM data and configure parameters easily.
2. Generate realistic traffic patterns with configurable parameters.
3. Find charging station locations that perform well in simulation.
4. Reproduce results consistently across multiple runs.

The ultimate goal is providing a reliable research and planning tool that helps optimizing EV charging infrastructure placement through data-driven simulation and analysis.

5 Research

5.1 Realistic Traffic Scenario Generation

From the beginning of the project, it was evident that the final simulation and placement of the charging stations would occur in SUMO. Therefore, the only question that arose during the research was whether the generation of the traffic model should happen within SUMO or a third-party framework.

Further research led to the conclusion that SUMO itself wasn't sufficient enough for generating realistic traffic scenarios. An own implementation or additional tool would be needed. The following describes the requirements, presents relevant simulation frameworks and further tools.

5.1.1 Requirements for Realistic Traffic Scenario Generation

To produce credible, repeatable traffic scenarios, the chosen tool or workflow must satisfy both core system requirements and specialized generation features.

- **R1.1 OSM-Integration:**

The system must integrate maps from OpenStreetMap (OSM) directly, ensuring that road layouts, intersection geometries and network attributes reflect real-world conditions. It is the foundation of any realistic traffic model.

- **R1.3 Configurable Parameters:**

Users should be able to adjust parameters like vehicle/ agent counts, agent behavior profiles and mobility patterns.

- **Traffic Agent Generation**

The system should automatically generate heterogeneous agents whose daily routines mirror real life (e.g., home-to-work, work-to-leisure, return home), with special attention to peak "rush-hour" flows and varying trip purposes.

- **R4.1 Automatic Conversion**

After importing raw OSM maps, the tool must convert them into a simulation-ready network format (e.g., SUMO's `.net.xml`, preserving all relevant attributes, speed limit, lane counts, turn restriction) needed for accurate vehicle routing.

- **R4.2 POIs** To enrich agent behavior and trip destinations, the system must detect points of interest (POIs) automatically from OSM data - such as residential areas, workplaces, schools, shopping centers - so that generated trips reflect plausible origins and destinations.

- **R5.1 Daily Patterns** In addition to origin-destination pairs, the synthetic population must adhere to a reasonable daily schedule (home → work → leisure → home) to ensure natural vehicle flow throughout the day.

- **R5.2 Temporal Variation** Because real traffic is never perfectly uniform the system should introduce stochastic variations in departure and arrival times around peak periods, so that

not all agents start or end trips at the exact same minute.

- **R5.3 Population**

Finally, the system must produce a complete population file ready for simulation (e.g., SUMO's `.pop` format) that encapsulates every agent's attributes, such as home and work locations, routines, and timing, without requiring the user to manually compile a list.

By implementing the above requirements, the system can generate traffic that is not only realistic but also reproducible.

5.1.2 Presentation of relevant tools

In recent years, many frameworks have been created and improved to support the development of realistic traffic simulations. In fact, it is now possible to copy entire cities and create agents that travel through them, visiting normal day-to-day points of interest. The following section will introduce tools and frameworks that can be used to generate realistic traffic from OpenStreetMap (OSM) maps.

SUMO

Simulation of Urban Mobility (SUMO) is an open-source, highly portable, microscopic, continuous traffic simulation package designed to handle large networks. It is developed at the *Institute of Transportation Systems of the German Aerospace Center* and released as the Eclipse SUMO project under the *Eclipse Foundation* **noauthor'sumo'nodate, noauthor'eclipse'nodate**.

It can simulate different types of road vehicles, including cars, buses, bikes, and pedestrians. It offers a wide range of tools to control the scenarios and data extraction. The core engine runs the simulation step by step, tracking each vehicle individually, enabling a detailed analysis of traffic dynamics, emissions and network performance **noauthor'eclipse'nodate**.

SUMO is compatible with many other tools. This enables the import of common network formats, such as OSM, *VISUM*, *VISSIM*, *OpenDRIVE*, and *MATSim* **noauthor'eclipse'nodate**. SUMO provides ready-made scripts that facilitate this process. These scripts make SUMO an extremely powerful tool for running the final simulation at the end of the workflow.

To help users get started with realistic road networks and traffic demand, it provides simple scripts and tools. One of these is the provided *osmWebWizard*. It is essentially a simple Python script that launches a web-based map interface. There, users can select any area from an OSM layer and set various parameters, such as the simulation duration, whether to include only vehicles, and the type of vehicles. The tool automatically downloads the underlying OSM data, converts it into a SUMO network, generates randomized traffic demand for the selected modes (e.g., cars and buses), and launches the SUMO GUI for immediate inspection **noauthor'osmwebwizard'nodate**.

In addition to the *osmWebWizard*, SUMO provides three other useful command-line tools, that might also be used for an implementation of an own workflow:

- `osmGet` enables bulk downloading of OSM data by geographic bounding box, named area (via OSM ID), or custom polygon files **noauthor'osmwebwizard'nodate**;
- `osmBuild` converts raw `.osm` files into SUMO's `.net.xml` format while applying custom filters, type mappings, and coordinate transformations **noauthor'osmwebwizard'nodate**;
- and `randomTrips` generates synthetic traffic by randomly sampling origin–destination pairs over time, with options for enforcing minimum trip distances, adding route prefixes, and repeating patterns at specified rates **noauthor'trip'nodate**.

SUMO already makes it easy to go from raw map data to a full traffic simulation—automating network import, demand generation, and scenario setup. However, its built-in traffic generation can be made even more realistic by pairing SUMO with specialized demand-modeling tools. Combined with SUMO's TraCI API, this hybrid approach allows experimenting not only with traffic scenarios but also with electric-vehicle charging plans, advanced control algorithms, and other complex workflows.

TraCI

TraCI (Traffic Control Interface) is SUMO's TCP-based application programming interface (API) that enables external applications to connect to a live SUMO simulation and interact with it in real time. By starting SUMO with a specified remote port, it becomes a server that waits for one or more clients to issue commands - advancing the simulation step by step, querying vehicle or lane data, charging states and vehicle speeds **noauthor'traci'nodate**.

To reduce communication overhead, TraCI supports a subscription mechanism that delivers bulk updates of selected variables, offering significantly higher throughput than individual polling. TraCI is available through bindings in Python, C++, Java, .NET, MATLAB and other languages, making it an extensible and high-performance interface for traffic simulation research **noauthor'traci'nodate**.

A striking example of TraCI's power is its ability to shade electric vehicles in real time according to their charging state - vehicles can automatically turn cyan when charging, red when their battery falls below a critical level, and green once sufficiently charged. This dynamic, in-simulation recoloring requires no simulation restart and provides immediate, intuitive visual feedback on EV behavior and infrastructure performance.

SAGA

SAGA (*SUMO Activity GenerAtion*) is an extension for the SUMO traffic simulator that automatically generates realistic, multi-modal mobility scenarios based on OpenStreetMap (OSM) data **codeca'saga'2022**.

SAGA essentially defines a clearly structured workflow that extracts all the necessary information from an OSM file. It then uses this information to generate all the configuration files for SUMO,

step by step. This includes road networks, public transportation stops and schedules, parking spaces, buildings, and points of interest. SAGA fills in missing data, such as parking space numbers or environmental features, with sensible default values that can be refined iteratively using external sources later on **codeca'saga'2022**.

Additionally, SAGA can generate activity-based travel plans upon request. It uses statistical or origin-destination (OD) data to create individual daily schedules (e.g., "home - work - shopping - home") incorporating various modes of transportation, such as walking, cycling, bus, or taxi. Activity chains, dwell times, and start times are drawn according to configurable distributions, and suitable route or inter-modal route algorithms in SUMO are employed. This allows even complex multi-modal commuter or leisure movements to be realistically mapped **codeca'saga'2022**.

The entire process is divided into individual, scriptable sub-steps, such as network setup, polygons, parking areas, TAZ weighting, OD matrix, activity definition, and timetable generation. This allows each phase to be adjusted and enriched independently **codeca'saga'2022**.

SAGA is freely available under EPL v2 on GitHub and has been integrated as a contributed tool since SUMO v1.3.0 **codeca'saga'2022**.

MATSim

MATSim is an open-source framework for large-scale, agent-based traffic simulations. It simulates a large number of individual travelers ("agents") on real roads and public transportation networks, supporting various modes of transportation, such as cars, public transportation, bicycles, and pedestrians. Thanks to its modular architecture and active community, MATSim is constantly being developed and can be adapted to specific requirements **community'matsim'2025**.

Simulations run dynamically, updating every second. Each agent attempts to optimize their daily routine (start and end times of activities, chosen route, and mode of transportation), but their decisions are always influenced by those of other travelers. Through an iterative optimization process, MATSim approximates realistic traffic flows and behavior patterns **community'matsim'2025**.

MATSim can map entire metropolitan regions with a high level of detail and second-by-second dynamics. The detailed outputs (e.g., vehicle and agent trajectories, KPI extraction) provide a solid basis for further analysis and visualization, whether for infrastructure planning, capacity analyses, or impact studies of mobility offerings **community'matsim'2025**.

BEAM

BEAM stands for behavior, energy, autonomy, and mobility. It is an open-source framework developed at *Lawrence Berkeley National Laboratory* and the *UC Berkeley Institute for Transportation Studies*. It is based on the well-known MATSim framework, but it adds a reinforcement learning approach, which is a type of machine learning. MATSim agents follow a daily schedule and only plan for the next day based on what has happened in the past. BEAM

agents learn as they go and can change their plans during the day to get the most out of it
noauthor'beamdocsaboutrst'nodate.

JOSM

JOSM is an extensible Java 11+ editor for OpenStreetMap. In addition to loading local data, it can be used to download selected data from online resources. It also allows users to create and modify nodes, ways, their relations, and metadata tags. JOSM is open-source and licensed under the GPL **noauthor'josm'nodate.**

5.1.3 Combination of SUMO with different frameworks/ tools

Since SUMO has been established as a fixed requirement for the placement and evaluation of charging station locations, it is possible to use SUMO exclusively or in combination with other tools for the realistic traffic generation. A few of these are explained below.

Combination of SAGA and SUMO

Since SAGA itself is a SUMO extension - so “combining” can sound redundant - it really provides a full, scriptable front-end that turns raw OSM data into a ready-to-run SUMO scenario. It handles everything from OSM import and network build through POI extraction, origin-destination-matrix (OD) generation, activity chain creation and route export, producing a complete `.sumocfg` that drops straight into SUMO.

SAGA generates highly realistic, activity-based daily schedules - enabling precise tuning of dwell-times, temporal variability (noise) and peak-hour dynamics. Its built-in multi-modal support automatically produces both vehicle flows and person-level population files for fully integrated simulations.

Outputs are immediately controllable via SUMO's TraCI API, enabling on-the-fly signal timing, rerouting or EV-charging logic without re-exporting.

Combination of MATSim and SUMO

The following flowchart shows how a combination of MATSim and SUMO could be implemented:

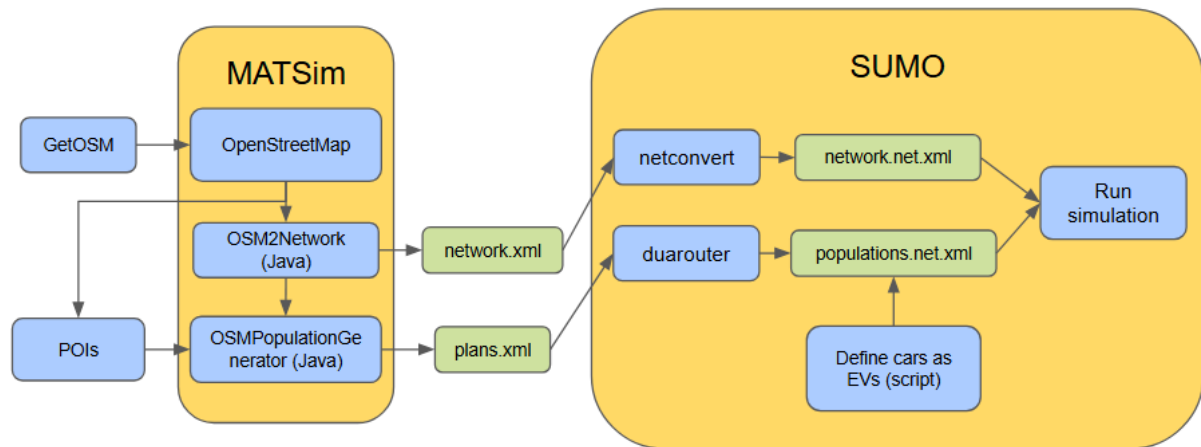


Abbildung 1: Integration of MATSim & SUMO

The process begins with acquiring raw OSM data via the `GetOSM` script provided by SUMO. Then, the OpenStreetMap data is entered into MATSim, where it is parsed to construct a MATSim-like network model. Next, the `OSM2Network` Java module converts the model into a SUMO-compatible `network.xml` file.

Meanwhile, the `OSMPopulationGenerator` creates an initial set of activity plans, which are serialized as `plans.xml`.

Once both files are produced, the workflow transitions to SUMO. The `netconvert` tool ingests the `network.xml` file from MATSim and outputs a SUMO-native `network.net.xml` file. Meanwhile, the `duarouter` tool takes the `plans.xml` file and generates the corresponding vehicle routes in the `populations.net.xml` file. A short post-processing script marks all passenger vehicles in the SUMO network as electric vehicles (EVs). Finally, the enriched network and routed population files are loaded into SUMO's simulation engine for execution.

Coupling MATSim's demand generator with SUMO's detailed simulation creates a modular, reproducible pipeline that handles both large-scale scenarios and individual vehicle behavior - ideal for EV studies. All intermediate files are archived, and MATSim strategies or SUMO driver models can be tuned independently. Key challenges might become the error-prone conversions of MATSim files into SUMO-compatible files.

Combination of BEAM and SUMO

In general, using BEAM and SUMO together would be very similar to using MATSim and SUMO. BEAM would also use the OSM file to create a MATSIM-like network and population. After conversion, the workflow would also label the vehicles as EVs and run the simulation.

However, the key difference is that SUMO acts as a low-level traffic engine with a feedback loop to BEAM. At each simulation step, BEAM pauses SUMO via TraCI. Then, it retrieves

vehicle states (position, speed, battery state of charge (SoC), and charger queues). After that, it updates agent decisions (energy consumption, charger detours, dynamic re-routing, and mode changes). Finally, it pushes those control commands back into SUMO.

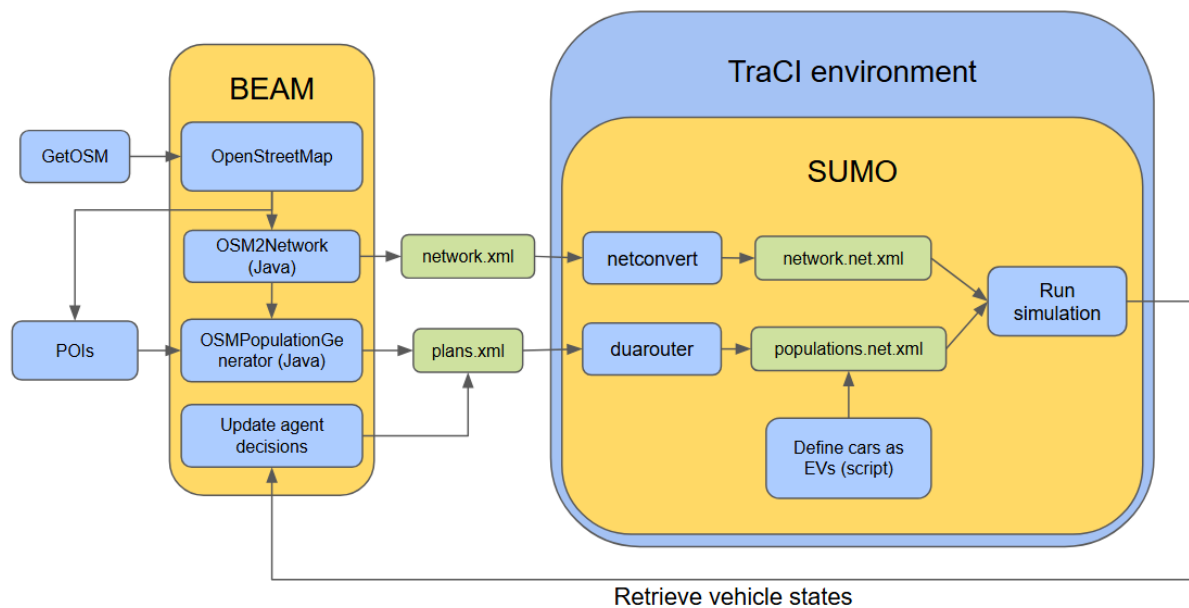


Abbildung 2: Integration of BEAM & SUMO

This tight, real-time exchange ensures charging events, congestion avoidance, and routing adjustments happen in real time. Rather than relying on a single, static routing pass, the simulation constantly "comes back" to BEAM to refine every vehicle's behavior.

Implementing a real-time feedback loop between MATSim and SUMO would add unnecessary overhead, since MATSim's day-to-day replanning already produces optimized plans that SUMO simply executes. A continuous Java to TraCI bridge would introduce extra inter-process messaging and callback wiring without yielding new insights.

5.1.4 Comparison of introduced concepts

The following section compares the technologies presented and possible combinations and evaluates them based on various criteria. Some of these criteria are quantitative, while others are based on personal trial and error and experience.

Criteria	SUMO	SAGA	MATSim+SUMO	BEAM+SUMO
OSM Import	✓	✓	✓	✓
Network Conversion	✓ via netconvert	✓ full OSM to SUMO pipeline	(✓) via OSM2Network (Java converter)	(✓) via OSM2Network (Java converter)
Demand Generation	(✓) randomTrips only, or custom scripts	✓ activity chains	✓ co-evolutionary plans	✓ activity & RL-learned
Temporal Realism	X (no built-in schedules, only with custom scripts)	✓ rush-hour noise + dwell	✓ dynamic daily replanning	✓ on-the-fly replanning
Multimodal Support	✓ cars, PT, bikes, pedestrians	✓ multi-modal by default	✓ multiple modes	(✓) mostly vehicles
Real-Time API	✓ TraCI	✓ TraCI	(✓) only in the SUMO stage (TraCI)	✓ tight feedback loop via TraCI + BEAM
Extensibility & Scripting	CLI, Python scripts	CLI + JSON configs + Python	Java APIs, Python scripts	Java + Python bindings
Documentation	Excellent docs	Not well-documented	Mature docs	Active docs
Flexibility	✓	✓	✓	✓
App Possibilities	general traffic sim	activity-based scenarios	research, large-scale planning	EV & energy system studies
Realism	basic realism	high realism	very high realism	very high realism
Performance	fast C++ core	Python/Java overhead	JVM + SUMO overhead	-
Complexity	moderate	higher setup complexity	quite complex	very high complexity
Notable Extras	osmWebWizard, osmGet/osmBuild	Auto-POI extraction; TAZ weighting	Large-scale OD matrices; plan-iteration	EV charging RL; dynamic rerouting

Tabelle 3: Comparison of SUMO, SAGA, MATSim + SUMO, BEAM + SUMO

SUMO is undoubtedly the most efficient and flexible option as a standalone simulation tool. The C++-based engine allows for high performance, and CLI commands and provided or custom Python scripts allow for large-scale network or behavior adaptations.

With the `osmWebWizard`, the entire workflow - from selecting and downloading the OSM map to setting attributes such as the simulation duration and finally starting the SUMO GUI - becomes easy.

However, there are no native mechanisms for generating demand beyond random trips and fixed daily or weekly cycles. Functions such as electric vehicles and on-the-fly replanning must be developed and integrated. While it offers more flexibility, it significantly increases the implementation effort and long-term maintenance requirements.

SAGA extends SUMO by providing a fully integrated OSM-to-SUMO pipeline that automatically extracts points of interest (POIs), allocates and weights traffic analysis zones (TAZ), and generates activity chains. Through this end-to-end preprocessing, it produces travel demand profiles with realistic daily patterns.

While the additional preprocessing stage introduces only a moderate performance overhead, SAGA remains fully open-source and can be customized via SUMO's TraCI interface to accommodate project-specific requirements. However, the existing user documentation contains significant gaps that hinder rapid onboarding, and built-in error-handling routines are currently insufficient for robust, large-scale applications.

MATSim and SUMO integrate the strengths of large-scale, agent-based demand modeling with a high-performance microscopic execution engine. Networks of large size can be imported directly from OpenStreetMap into MATSim, where optimized agents' daily plans are generated based on activity schedules. These precomputed plans are then executed in SUMO, leveraging its performant core for vehicle kinematics.

During runtime, the TraCI interface enables dynamic interventions — vehicles can be rerouted in response to emerging congestion, and battery state-of-charge (SoC) can be monitored and controlled for electric-vehicle fleets. Nevertheless, it should be noted that most of the routing logic is done in MATSim and SUMO itself only acts as a fast execution layer. This separation can limit the granularity of on-the-fly adaptations and may introduce additional complexity.

In addition to traffic simulation, BEAM integrates energy systems and offers electric vehicles (EVs) and charging infrastructure. However, the real-time loop between SUMO and BEAM, as well as the stepwise simulation, introduces massive overhead. This, coupled with the complex implementation, may make this combination a less attractive option for many projects.

5.2 Charging Station Placement Algorithm

5.2.1 Requirements for Charging Station Placement Algorithm

The placement algorithm must first perform clustering to detect EV-demand hotspots and then automatically propose new charging-point candidates. It should support exporting its findings in GeoJSON and XML, and use a multi-criteria optimization engine — balancing traffic flow, site accessibility and grid capacity to select locations. The system must be able to optimize toward different objectives, such as maximizing coverage or minimizing average user distance, and rank all proposed sites with a scoring metric. Finally, it should maintain detailed logs of every optimization run for auditing and analysis.

Another aspect for placement is the consideration of a self-owned in-house wallbox or charging options at the work location, which a person might be more likely to use **ev`cs`planning`with`dyn`pred.** This can be explained by the trip chain, which is consistent with people's daily travel law. Eventually, the placement and removing should be executed automatically, based on traffic, traffic flow, based on a realistic simulation **cs`ev`in`SUMO`impact`on`traffic`flow.**

5.2.2 Assumptions

Parking areas

There are two main reasons why implementing realistic “parking-bay” chargers in SUMO is exceptionally difficult. First, SUMO's network model is built around edges and lanes rather than individual parking stalls. In order to represent individual parking spots, each lot would need to be divided into dozens or hundreds of tiny edge segments and assigned its own links. Additionally, traffic lights or way logic would have to be implemented to prevent collisions, and a custom script would have to be created to handle the routing of EVs so they can actually park in those bays. This would greatly increase development time and the risk of errors.

Second, SUMO's electric-vehicle extension only supports chargers attached to lanes or edges. There is no built-in mechanism for off-lane or stationary charging.

For these reasons, we assume that EVs charge in a special “charging lane” rather than in separate parking bays. Vehicles simply pull into that lane; SUMO's existing battery and charger modules work unmodified; and we avoid custom network-generation scripts while still capturing the key effects of charging stops on routing and traffic flow.

Points of Interests

Certain areas can be considered points of interest. Namely, work location, shopping locations, residential areas or home addresses, to be more specific and others, that qualify. It is reasonable to believe, that the extraction of points of interest from the OSM will offer a wide range of potential improvements to the system. For example when trying to optimize traffic and maybe even go so far, to say that cars can and should be assigned specific charging stations, that potentially are not the closest, but most optimal - considering waiting time, distance, travel time and other

factors **wevj14020040**. Currently points of interest have no inherent value or use, when looking at the documentation of SUMO itself, "definitions of colored polygons and POIs (points of interest) can be loaded within an additional-file. These shapes are currently meant to improve a simulation's appearance and to allow an easier debugging. No special interaction with them is implemented, yet"**sumo`shapes**.

5.2.3 Charging Station Algorithm

The rationale behind the Charging Station Placement Algorithm is primarily to identify optimal locations for charging stations (CS) that reliably capture high traffic volumes and minimize waiting times. To emphasize the importance of traffic patterns, Figure 3a presents a heatmap of heavily used roads, while Figure 3b shows the residential origins of travelers, which helps explain certain red edges in the road heatmap. Naturally, placing CS near roads with high vehicle throughput is a sensible approach, as most drivers are likely to pass through these areas. However, exploring ways to better distribute that traffic may be even more effective in reducing waiting times.

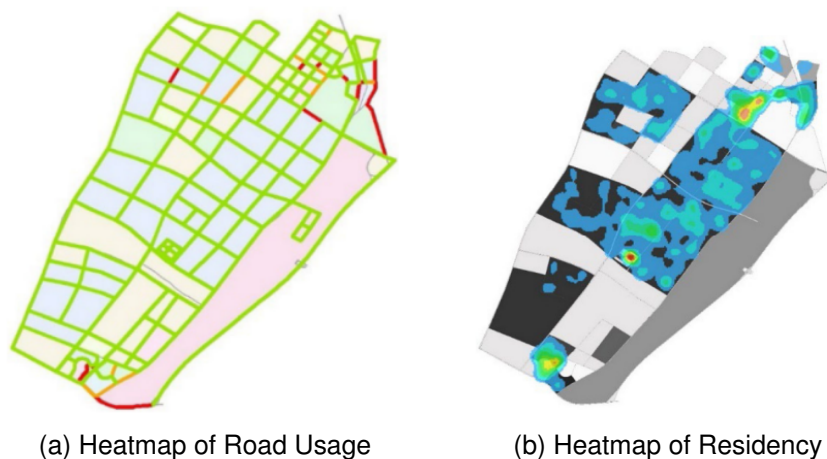


Abbildung 3: Comparison of Heatmaps for Roads and Residency

Due to the objective, an iterative approach seems well-fit to satisfy the requirements. The CS will be randomly placed, as it cannot be anticipated how the lanes are built, before the first iteration of the simulation and after it has finished, the data will be logged to extract information from, which will be used to improve the existing system. Certain locations, such as remote areas along highways, may not be considered for charging station placement if they are unlikely to reach the required usage levels. To combat that potential error, implementing machine learning could be a big step, to account for uncertainty, improve positioning of CS with actual traffic flow, density and occupancy of roads themselves **vazifeh2015optimizing**.

5.3 Integration of Grid Models

5.3.1 Requirements for Integration of Grid Models

For the efficient operation of vehicle-to-grid (V2G) systems, it is essential to integrate grid-relevant data into the optimization process for the charging infrastructure. As already described in the requirements above, the locations of charging stations should also be optimized in relation to existing grid capacity. Grid conditions should be taken into account in order to reduce grid loads and avoid charging peaks. This is to ensure that charging points are not only user-friendly but also positioned in a way that is compatible with the grid.

5.3.2 Approaches to integrate V2G functionality

SUMO itself does not natively support direct integration for connecting or simulating electrical networks. Therefore, a third-party or proprietary implementation is necessary. The following describes possible approaches that could implement V2G functionality, but these will be further validated in upcoming research phases.

Integration of Power Lines into Simulation Network

Although SUMO does not offer native support for power grids, power lines can still be modeled. This can be done, for example, using the `netedit` tool provided by SUMO. The power lines are integrated into the existing road network as special edges with the attribute `type="powerline"`. In this way, both fictitious and real power grids can be integrated into the simulation and visualized.

To evaluate the location of a planned charging station, Python scripts can be used to check whether the station is sufficiently close to a power line using geometric calculations. If the distance is below a defined threshold (e.g., 50 meters), the location can be considered suitable - otherwise, it is rejected.

This method serves as an efficient pre-filter when planning charging infrastructure, especially for large-scale networks or urban scenarios.

Runtime Monitoring of Grid Loads

SUMO maps the charging process of electric vehicles, but does not simulate physical loads or capacities of the power grid. In order to evaluate a charging station during runtime based on grid capacity, SUMO could be extended with internal control logic via a TraCI interface.

This would involve implementing a state memory for each charging station during runtime, which would record how many EVs are charging at any given time, how much electrical power is being drawn, and whether the station is considered overloaded. In vehicle-to-grid scenarios, it would also be possible to document whether vehicles are feeding energy back into the grid.

This data enables a detailed analysis of the temporal load distribution at individual stations. This allows overload situations to be detected, not often used stations to be identified, and possible

measures for optimizing performance, number, or location to be derived.

An extended version of this approach involves linking to an external grid model, e.g., via `pandapower`. The charging stations are mapped as nodes (buses) in the electrical grid. A network calculation is performed for each new charging request **noauthor'pandapower'nodate**, **noauthor'about'nodate**. If the additional consumption leads to overvoltage or overload, the charging process can be specifically prevented in SUMO or rejected by the station.

This approach would make it possible to take into account not only the location of charging stations, but also the technical limitations of the power grid.

6 Decision and implementation

6.1 Decision

For the first prototype implementation, the combination of MATSim and SUMO was chosen in order to leverage the strengths of both tools. MATSim offers scalable, agent-based demand generation, allowing realistic daily schedules and routes to be implemented based on activity patterns and POIs. SUMO, on the other hand, is a high-performance microscopic traffic simulator that can accurately map vehicle movements.

By combining both tools, the generation of agents and routes (MATSim) and the actual simulation (SUMO) are separated. In addition, SUMO can be controlled by TraCI during runtime. This creates a modular and scalable system architecture that effectively represents electric mobility scenarios.

Other approaches described, such as BEAM and SAGA, were initially rejected. BEAM proved impractical due to the high complexity of the feedback loop caused by replanning and the large overhead. Although SAGA offers a very promising pipeline and was also tested, it was also ruled out due its insufficient documentation and flawed error handling at the the time of the project.

6.2 Implementation

The following section describes the first prototypical workflow and the transition to the current implementation. Initially, agent-based daily schedules and travel routes were generated in MATSim, while microscopic traffic simulation was subsequently performed in SUMO. However, during development and especially during stress testing, this approach proved to be error-prone: There were repeated problems converting MATSim networks and routes into SUMO-compatible formats, which meant that simulations could not be run with the generated data.

These difficulties ultimately led to a switch to the current workflow, in which the routes are now generated entirely by our own Python scripts independently of MATSim.

6.2.1 Prototyping

The project's initial design and vision were promising because the tools offer powerful options for creating simulations. Coupling MATSim's demand generation with SUMO's microscopic simulation created a modular, flexible workflow well-suited for electric mobility scenarios at large and detailed scales.

The figure above illustrates the structure of the initial workflow. A built-in integration of SUMO's `osmGet` script retrieves raw OSM data, although alternative tools like JOSM could be used as well. This data is then converted by two developed MATSim functions into a `network.xml` file, along with a `plans.xml` file generated from the provided points of interests (POIs). The

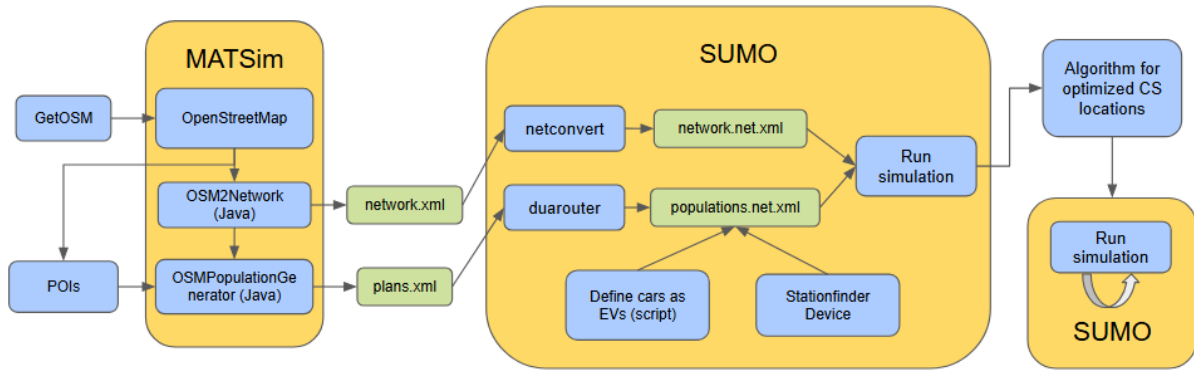


Abbildung 4: Flowchart of Prototype Design

`plans.xml` contains the travel routes agents will follow between their POIs.

Next, the system converts these files into SUMO-compatible formats using `netconvert` and `duarouter`, which process the network's nodes and edges. After successful conversion, all vehicles defined in the `population.net.xml` file are overwritten as electric vehicles (EVs) and equipped with a `stationfinder` device. This device allows them to dynamically reroute to nearby charging stations if needed, deviating from their original paths.

The simulation is then executed, and an optimization algorithm analyzes the logged results to improve the placement of charging stations. The updated station locations serve as input for the next iteration - forming a optimization loop that refines the infrastructure with each cycle.

The placement and optimization of charging stations has not yet been implemented in this prototype workflow, as the conversion from MATSim to SUMO already led to errors.

6.2.2 Refactoring and technical learning

As described above, the combination of MATSim and SUMO proved to be too error-prone to continue. This resulted in a refactoring of the project, during which previously developed Java classes and dependencies were removed. This step significantly reduced complexity, as the Java-based structure had introduced overhead and rigidity.

Despite its limitations, the initial prototype workflow was instrumental in building a deeper understanding of SUMO's simulation architecture, including the required input files, data formats and configuration files. This foundation greatly accelerated the development of the further implementations.

6.2.3 Current implementation

The current implementation builds on the experience gained with the first prototype, but completely replaces MATSim. Demand generation is now performed by proprietary scripts in pre-processing and flows directly into the subsequent optimization.

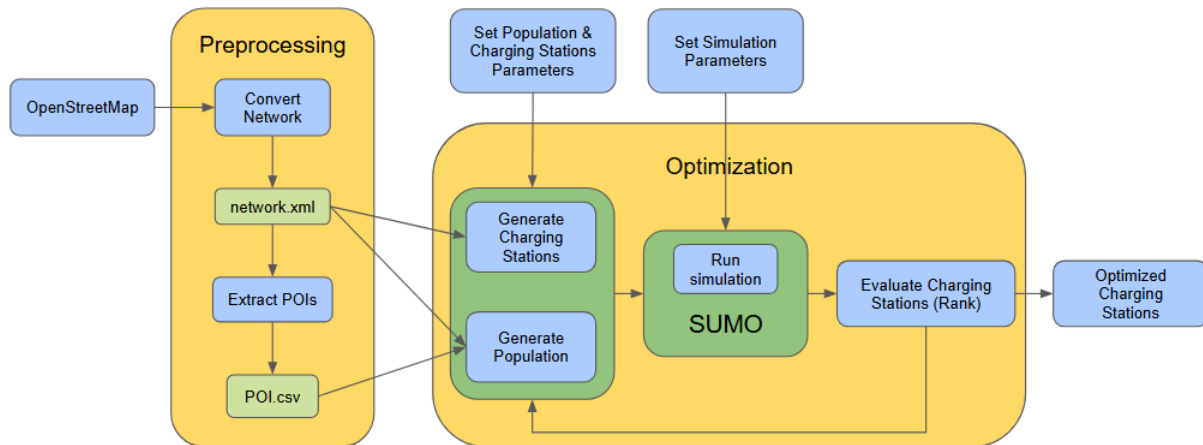


Abbildung 5: Flowchart of Creating and Running a Simulation

As in the previous implementation, an OSM file forms the basis of the entire simulation. This is downloaded using the `osmGet` tool and serves as the foundation for generating and converting the traffic network (`network.xml`). In addition, in this implementation, the POIs are extracted directly from the OSM file. Filtering is performed using relevant attributes such as `amenity="residential"`, `office`, `shopping`, etc., with the results being saved in a CSV file (`POI.csv`). This step completes the preprocessing state.

The system then generates an initial population and initial distribution of charging stations based on the previously generated files (`network.xml`, `POI.csv`) and user-defined parameters. The SUMO simulation can then be started immediately, as no additional conversion steps are required - which is the main advantage of this implementation in comparison to its prototype and significantly reduces the likelihood of errors.

In the subsequent optimization phase, an algorithm works to iteratively improve the placement of charging stations. At the beginning, for example, charging stations are provisionally placed on all road sections (lanes) with a length of more than 70 meters. After a simulation run, the number of vehicles that have charged at each station is recorded. This information is stored in a log file and evaluated. In each iteration, the number of charging stations is then reduced until only a defined target number of locations remains.

The result of this workflow is an optimized distribution of charging stations based on actual usage, which can be adopted for final deployment.

6.2.4 Overview of architecture and Dataflow

As can be seen in the workflow shown above, the implementation comprises three central functionalities:

- The generation of trips based on the OSM file and the POI coordinates,
- the creation of charging stations on suitable lanes,
- and the optimization algorithm for simulation, evaluation and placing the charging stations.

These functions are identified in the project structure by the files `mainGenerateTrips.py`, `mainGenerateChargingStations.py`, and `mainSim.py` and are visualized again in the following diagram. The figures also show how the files generated by the first two functions are merged in the simulation.

The three sub-functions are described in detail below.

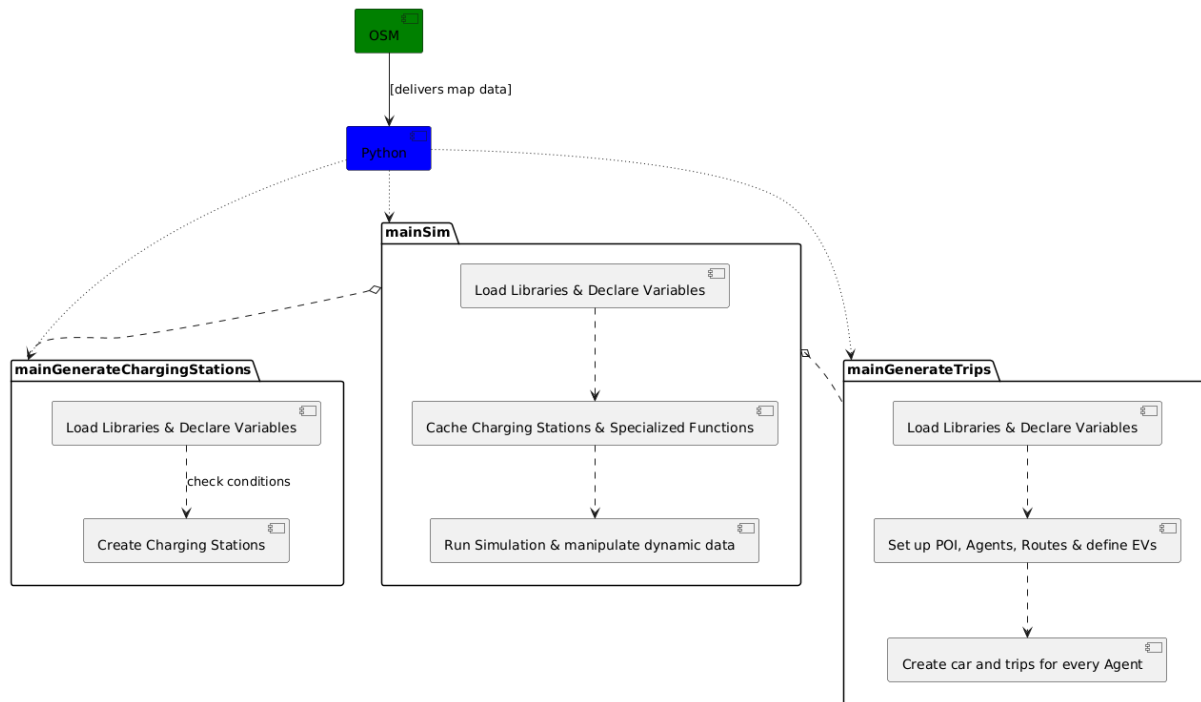


Abbildung 6: Abstracted View of Architecture

Population Generation

The population or trips that the agents follow during simulation are generated automatically from the network and predefined simulation parameters. These include points of interest, the number of desired agents, and the proportion of EVs.

The following activity diagram provides a simplified description of the generation process. After loading the SUMO network, the edges from the `POI.csv` that are suitable for car traffic are first validated and filtered. If no edges are valid, the script is aborted to avoid invalid routes.

Then, a loop starts to create agents, i.e., drivers who are to move between POIs on the map. Random home and work edges are selected for each person. In addition, a predetermined proportion is used to decide whether the person will receive an electric vehicle or a conventional vehicle (combustion engine).

In a second loop, an individual vehicle route is created for each person. This includes a random departure time in the morning within the typical rush hour (7-9 a.m.) and an eight-hour stop at the workplace. After route planning, all vehicles are sorted chronologically. The sorting must be done based on the sequential process of SUMO, because it processes each trip one after the

other – from top to bottom.

Finally, all vehicles, including their routes and stops, are written to an XML file. Vehicle-specific parameters such as battery capacity and energy behavior are set. The result is a complete file that can be used directly for a SUMO simulation without conversion.

Without this custom script, other frameworks that have already been partially introduced would have to be used to generate the trips, which would increase overhead and the likelihood of errors. With this custom implementation, scenarios can be generated effectively and reliably, completely replacing MATSim.

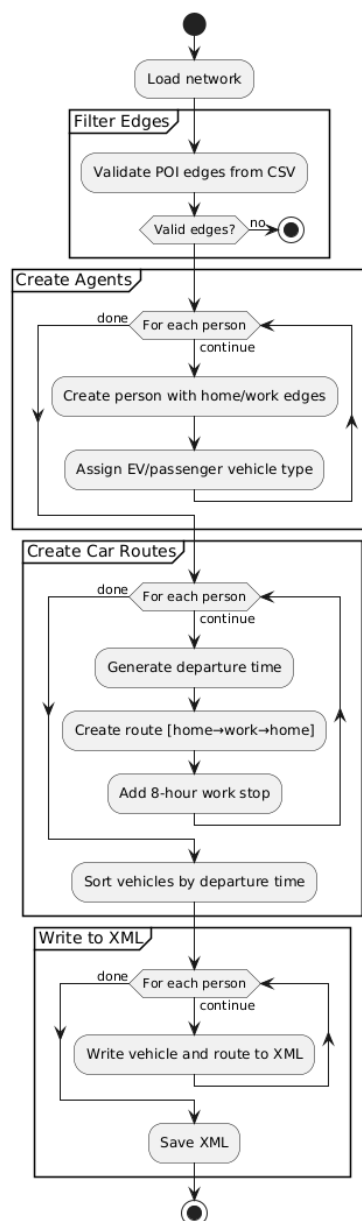


Abbildung 7: Trip Generation Activity Diagram

Charging Station Generation

Just like the trips, the initial charging stations are inserted via a script. The following diagram briefly describes this process.

First, the network is loaded, considering only roads that can actually be used by vehicles. Each of these edges is then validated again. If it fails validation, it is skipped.

Next, all lanes of these valid roads are iterated through. For each lane longer than 70 meters, the center is calculated, a unique ID is generated, and a `chargingStation` element is created at that location. Each charging station receives fixed values like power capacity and `chargeInTransit`. The last one restrains cars from charging in active traffic and forces them to park, while doing so.

All generated elements are then used to build an XML structure, which is written to an XML file. This file can be used in the SUMO simulation without any conversion - just like the trips.

It is important to note that these charging stations represent only initial locations, which may be removed during the optimization process by the developed algorithm, for example, if they are not used by EVs.

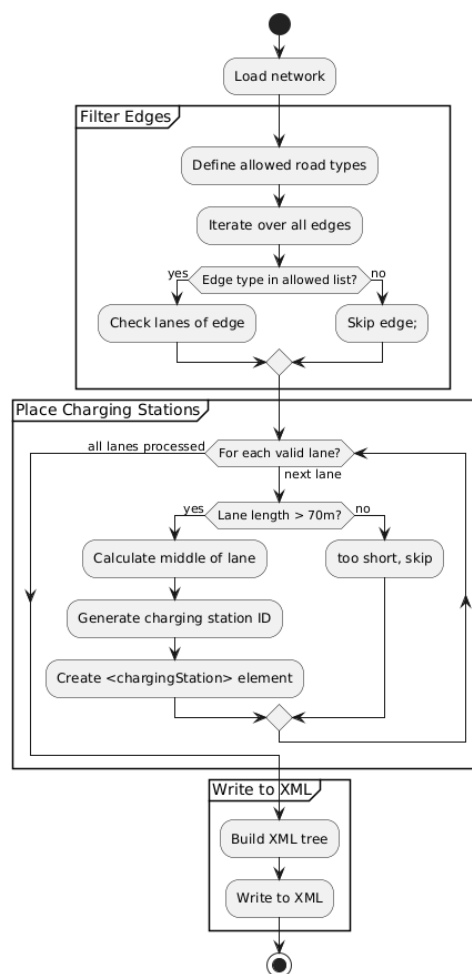


Abbildung 8: Charging Station Deployment Script Activity Diagram

Simulation

While SUMO acts as the "heart" of the simulation - executing vehicle movements based on preloaded data - TraCI functions as its "brain", providing the ability to dynamically alter the simulations during runtime. Although SUMO could definitely run with only static XML files, TraCI enhances it by allowing real-time control over nearly every simulation aspect. Because of TraCI it is possible to dynamically route and change attributes while using a high level programming logic.

The following activity diagram illustrates an abstracted view of how the simulation is being initialized and how EVs are tracked during the simulation using TraCI.

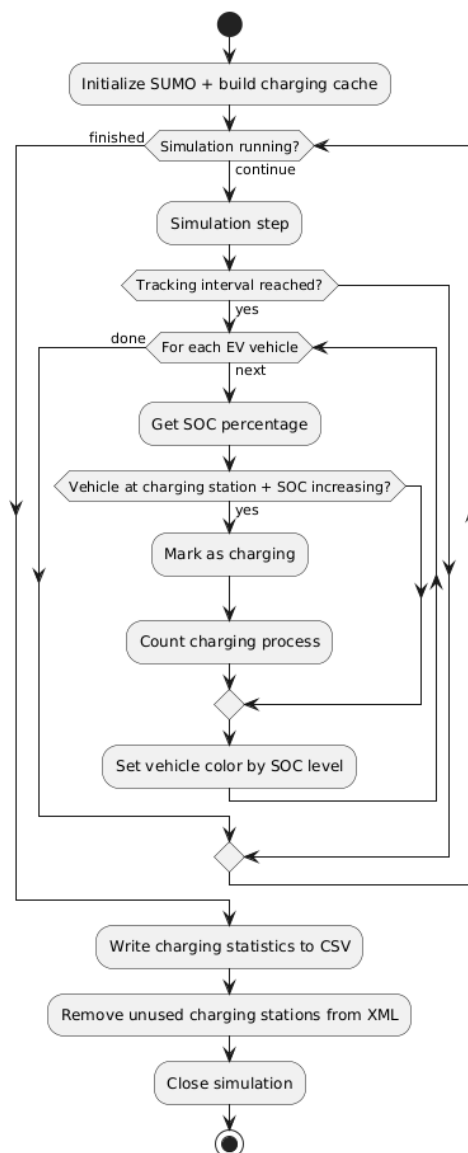


Abbildung 9: System Activity Diagram

At the beginning, the system finally initializes the SUMO simulation - either as GUI or CLI application - and builds a cache of all charging station positions. This cache contains coordinates

of all charging stations and improves lookup times during runtime.

Once initialized, the simulation enters its main loop, which continues to run as long as there are active vehicles. Each iteration the simulation advances by one time step.

At specific intervals, which are defined by a tracking interval constant - the system performs deeper analysis. During these intervals, all known EVs are evaluated individually. For each EV, the system retrieves its current state of charge (SOC) in percentage and colors them accordingly.

If a vehicle is found to be stationary at a charging station and its SOC is observed to be increasing, it is marked as actively charging, and the counter for this specific charging station next to it increases. This is important because the optimization algorithm will use it to determine the usage and ranks of the charging stations.

Regardless whether the vehicle is charging, its color is updated based on their SOC level. This visual feedback helps monitoring battery status in real-time within the SUMO-GUI.

Once the simulation loop ends, so all vehicles have left the network (are not active anymore), the system writes a summary of the charging statistics to a CSV file.

CS_id	charges
CS_437325054#0_0	7
CS_1100889498_0	4
CS_492340367#1_0	4
CS_541295978#1_1	3
CS_437325054#0_1	1
CS_-6228081#1_1	0

Tabelle 4: CS_id and recorded charges per CS

Finally, based on the recorded usage data, the system removes underused or unused charging stations from the XML configuration. The simulation is then properly closed and can be run again in the next iteration.

CS_id	charges
CS_437325054#0_0	7
CS_1100889498_0	4
CS_492340367#1_0	4

Tabelle 5: CS_id and recorded charges per CS after removing underused CS stations

This workflow enables dynamic tracking of EV charging behavior and allows post-simulation filtering of charging infrastructure. It can be repeated as long as the user wishes for or the number of charging stations doesn't decrease anymore.

7 Results

7.1 Showcase

The following graphic shows a compilation of a running simulation in SUMO. The colors indicate the charge level of the vehicles and whether a vehicle is currently at a charging station and charging.

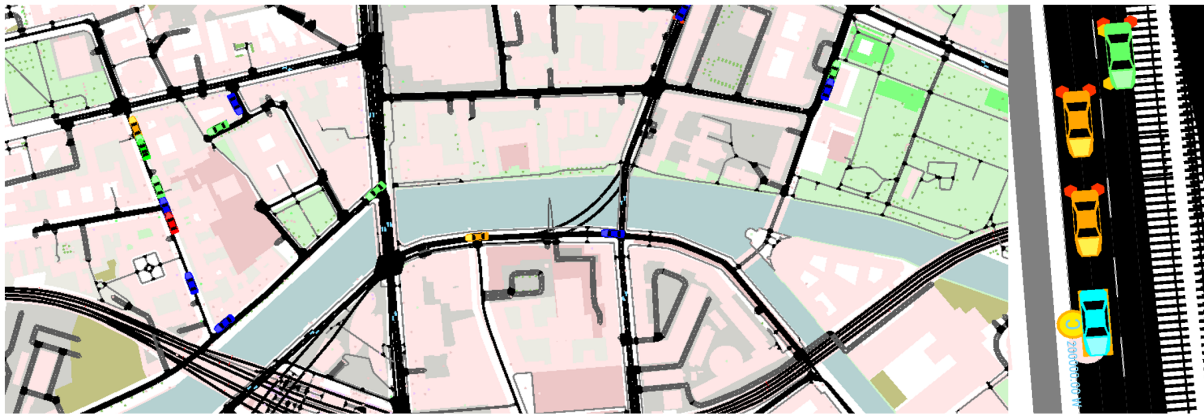


Abbildung 10: Showcase of current implementation

As described above, the charging stations were created as non-blocking lanes on the actual lanes. This means that vehicles charging there can charge without holding up the flow of traffic.

The vehicles drive to their POIs during rush hour. For example, in the morning they will drive to work between 7 and 9 a.m., stay there for an average of 8 hours, and then continue on to the next POI or back home. They follow their predefined routes. TraCI reads the current charge level of the vehicles and colors them in the `sumo-gui` depending on their SoC. This facilitates qualitative evaluation.

If a vehicle's charge level falls below 20 percent of charge on its route, the implemented `stationfinder` ensures that the vehicle drives to the nearest charging station, where it is then charged. Once fully charged, the car then continues on to its originally intended POI.

7.2 Functionalities

The central functional modules of the first iteration that have already been implemented are listed and described in more detail below.

Functionality	Description	Inputs	Outputs	Acceptance Criteria
OSM Import				
Selection & download of OSM data	User selects a map area; system runs <code>osmGet</code> and saves the <code>.osm</code> file into the project directory.	Map selection	<code>.osm</code> file	<code>.osm</code> present in target folder
Convert OSM to SUMO network	Transforms the <code>.osm</code> file into a SUMO <code>network.xml</code> via <code>netconvert</code> .	<code>.osm</code> file	<code>network.xml</code>	Loads without errors in SUMO GUI
Traffic Generation				
POI extraction	Parses <code>.osm</code> , filters by amenities (residential, commercial, school, etc.), and exports a geocoded CSV.	<code>.osm</code> file	<code>poi.csv</code>	POIs/amenities appear with valid lat/long entries
Trips generation	Generates <code>trips.xml</code> for N agents using map data and POI locations.	<code>.osm</code> , <code>poi.csv</code>	<code>trips.xml</code>	Agents assigned valid origin/destination
Automatic rerouting	Monitors vehicle SoC via TraCI and reroutes vehicles below threshold to nearest charger.	SoC threshold; charger list	Updated route assignments	Low-SoC vehicles rerouted within 30 s of threshold hit
Charging Station Placement				
Auto-place charging stations	Identifies lanes $\geq L_{\min}$ and assigns each a charger coordinate.	Minimum lane length $L_{\min}$	Charger coordinate list	100% of lanes $\geq L_{\min}$ receive one charger placement
Rank charging stations	In simulation, increment station score on each use; output a sorted list by usage frequency.	Simulation usage logs	Ranked station list	Ranked list of stations with number of all charge events
Iterative reduction of stations	Removes lowest-ranked chargers until only the top-K remain; outputs optimized placement.	Ranked station list	Optimized charger list	Top ranked stations are placed in simulation
Simulation				
<i>Continued on next page</i>				

Functionality	Description	Inputs	Outputs	Acceptance Criteria
Color vehicles by SoC	Via TraCI, color vehicles in GUI based on current SoC or charging status.	SoC thresholds; color mapping rules	Real-time colored visualization	Colored vehicles
Performance				
Caching	Cache charger coordinates in memory to reduce repeated TraCI lookups and speed up simulation loops.	–	–	Simulator runtime reduced, no lags

8 Discussion

8.1 Interpretation of Results and Evaluation of Workflows

The research and first implementations have demonstrated that it is possible to generate realistic traffic scenarios from OpenStreetMap data and optimize charging station locations based on demand. Two main workflows were developed, tested and evaluated in detail: MATSim combined with SUMO, and SUMO as a standalone tool.

While the combination of MATSim and SUMO showed initial promise, it ultimately proved unreliable. The most significant barrier was the conversion of networks and trips between the two tools. SUMO's provided Python script frequently failed to convert MATSim generated `.xml` files, primarily due to node and edge mismatches. This made it impossible for agents to follow their predefined routes, effectively hindering the whole simulation. After multiple iterations and time-consuming troubleshooting, it was concluded that MATSim was not suitable for integration with SUMO in the current setup and had to be disregarded.

Similarly, BEAM - an extension of MATSim - offers promising features for vehicle-to-grid (V2G) applications. However, it inherits the same limitations regarding conversion, and SUMO does not provide any direct conversion tools for BEAM. That being said, its practical usability remains questionable in this context.

SAGA was briefly considered during the initial development phase. However, because of its limited documentation and persistent errors, the progress was very slow. Due to these challenges, it was also excluded from further evaluation.

By contrast, SUMO as a standalone tool proved to be powerful and reliable for urban microscopic traffic simulations. Its extensive collection of scripts for traffic generation and statistical analysis, combined with the flexibility of TraCI, provides a solid and customizable foundation. Well-documented functions and tools such as `osmGet` and `osmWebWizard` significantly reduce onboarding difficulties and make it an excellent base for further development.

8.2 Optimizing Charging Station Placement

One of the key objectives of the project is the optimization of charging station placement. TraCI, SUMO's interface for traffic control was effectively used for data collection during runtime. The currently implemented algorithm - based on iterative tracking, ranking and pruning of charging stations - served as a good initial approach. However, this method can be improved by introducing more dynamic and predictive techniques such as machine learning.

In addition, the pruning process, where underused charging stations are deleted from the simulation, could remove too many stations and therefore lack adaptability. Future improvements should reintroduce stations based on variables like demand, station density and inter-station distance. This could create better balance between efficiency and coverage.

8.3 Challenges with Power Grid Integration

A major yet unaddressed challenge is the integration of power grid data to enable V2G simulations. The possible solutions outlined above were not incorporated into this prototype but will be essential for advancing bidirectional charging and modeling the interaction between EVs and the energy network. Furthermore, integrating grid data would allow for more strategic placement of charging stations based on their connectivity to the power infrastructure. This remains a key aspect for future development.

8.4 Fulfillment of Research Objectives

Looking back at the research objectives, all key objectives defined across the first three project phases were successfully archived.

In Phase 1, functional and non-functional were identified, and relevant methods and data sources were explored, resulting in first prototypical concepts and workflows.

During Phase 2, a working prototype using SUMO and TraCI was developed. A realistic traffic scenario was created, an algorithm for optimal charging station placement was implemented, and initial stations were placed within the simulation.

In Phase 3, the prototype was iteratively validated. The results confirmed the effectiveness of the placement strategy and highlighted areas for future refinement.

Overall, the project fulfilled its main goals and provides a strong foundation for continued development, particularly in the areas of V2G integration and optimization.

8.5 Further work

Future work should build upon the current implementation by addressing the following key areas:

- **Generalizing the workflow for any area**

The current system should be adapted to work across different geographic regions with varying infrastructure and demand profiles. Automating data import from OSM and standardized parameters will be essential for broader applicability.

- **Interactive Dashboard**

An user-friendly dashboard would allow users to select arbitrary regions, adjust parameters (e.g. station count, vehicle types), visualize simulation outputs and monitor charging behavior in real time.

- **Leveraging WebWizard**

SUMO's `osmWebWizard` should be cleverly included into the workflow. Integrating it would reduce the complexity and enable a faster development.

- **Modeling Power Grid Interaction**

A realistic energy grid model must be incorporated to simulate load distribution, feedback from EVs, and the impact of bidirectional charging.

- **Optimization**

The placement algorithm should be enhanced with data-driven or machine learning approaches that can adapt to more complex demand patterns.

- **Improved Scenario Generation**

Alternatives to the current workflow, such as implementation with BEAM or SAGA, should be explored further ensuring a more realistic and large scale demand pattern.